LAB 09:

Question#01:

Code:

```cpp
#include <iostream>
using namespace std;

// Abstract base class
class ArrayMultiplier
{
public:
    virtual void calculate() = 0; // Pure virtual function
};

class ArrayMultiplier1D : public ArrayMultiplier
{
    int arr[10];
    int size;

public:
    ArrayMultiplier1D(int a[], int n)
    {
        size = n;
        for (int i = 0; i < size; i++)
            arr[i] = a[i];
    }

    void calculate() override
    {
        int result = 1;
        for (int i = 0; i < size; i++)
            result *= arr[i];
        cout << "1D Array Multiplication Result: " << result << endl;
    }
};

class ArrayMultiplier2D : public ArrayMultiplier
{
    int arr[10][10];
    int rows, cols;
```

```
public:
    ArrayMultiplier2D(int a[][10], int r, int c)
    {
        rows = r;
        cols = c;
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                arr[i][j] = a[i][j];
    }

    void calculate() override
    {
        int result = 1;
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                result *= arr[i][j];
        cout << "2D Array Multiplication Result: " << result << endl;
    }
};

int main()
{

    int a[] = {1, 2, 3, 4};
    ArrayMultiplier1D obj1(a, 4);
    obj1.calculate();

    int b[2][10] = {{1, 2, 3}, {4, 5, 6}};
    ArrayMultiplier2D obj2(b, 2, 3);
    obj2.calculate();

    return 0;
}
```

Output:

```
6_Q1 }
1D Array Multiplication Result: 24
2D Array Multiplication Result: 720
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 09>
```

Question#02:

Code:

```cpp
#include <iostream>
using namespace std;

// Abstract base class
class Store {
protected:
    double total_bill;

public:
    Store(double bill) {
        total_bill = bill;
    }

    virtual void calculateFinalBill() = 0; // Pure virtual function
};

// Derived class for ImtiazStore (7% discount)
class ImtiazStore : public Store {
public:
    ImtiazStore(double bill) : Store(bill) {}

    void calculateFinalBill() override {
        double discount = total_bill * 0.07;
        double final_bill = total_bill - discount;
        cout << "ImtiazStore - Total Bill: " << total_bill << endl;
        cout << "ImtiazStore - Discount (7%): " << discount << endl;
        cout << "ImtiazStore - Final Bill: " << final_bill << endl;
    }
};

// Derived class for BinHashimStore (5% discount)
class BinHashimStore : public Store {
public:
    BinHashimStore(double bill) : Store(bill) {}

    void calculateFinalBill() override {
        double discount = total_bill * 0.05;
        double final_bill = total_bill - discount;
        cout << "\nBinHashimStore - Total Bill: " << total_bill << endl;
        cout << "BinHashimStore - Discount (5%): " << discount << endl;
```

```
        cout << "BinHashimStore - Final Bill: " << final_bill << endl;
    }
};

int main() {
    ImtiazStore store1(5000);
    store1.calculateFinalBill();

    BinHashimStore store2(5000);
    store2.calculateFinalBill();

    return 0;
}
```

Output:

```
6_Q2 }
ImtiazStore - Total Bill: 5000
ImtiazStore - Discount (7%): 350
ImtiazStore - Final Bill: 4650

BinHashimStore - Total Bill: 5000
BinHashimStore - Discount (5%): 250
BinHashimStore - Final Bill: 4750
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 09>
```

Question#03:

Code:

```
#include <iostream>
#include <string>
using namespace std;

// Abstract Base Class
class Vehicle {
protected:
    int carId;
    string brand;
    string model;
```

Object Oriented Programming:

```
public:
    Vehicle(int id, string b, string m) {
        carId = id;
        brand = b;
        model = m;
    }

    virtual bool isAvailable() = 0; // Pure virtual function
    virtual void rent() = 0;        // Pure virtual function
    virtual void returnVehicle() = 0; // Pure virtual function

    void displayInfo() {
        cout << "Car ID: " << carId << " | Brand: " << brand << " | Model: " <<
model;
    }

    virtual ~Vehicle() {} // Virtual destructor
};

// Derived Class: Car
class Car : public Vehicle {
    bool available;

public:
    Car(int id, string b, string m) : Vehicle(id, b, m) {
        available = true; // Car is available by default
    }

    bool isAvailable() override {
        return available;
    }

    void rent() override {
        if (available) {
            available = false;
            cout << "Car rented successfully -> ";
            displayInfo();
            cout << endl;
        } else {
            cout << "Car not available -> ";
            displayInfo();
            cout << endl;
        }
    }
```

Made By: Faarid Amir Shaikh CT-25276 Section: F

Object Oriented Programming:

```
    void returnVehicle() override {
        available = true;
        cout << "Car returned successfully -> ";
        displayInfo();
        cout << endl;
    }
};

// Rental System Class
class RentalSystem {
public:
    void rentVehicle(Vehicle* v) {
        if (v->isAvailable()) {
            v->rent();
        } else {
            cout << "Sorry, this vehicle is not available for rent!" << endl;
        }
    }

    void returnVehicle(Vehicle* v) {
        v->returnVehicle();
    }
};

// Customer Class
class Customer {
    string name;
    RentalSystem rs;

public:
    Customer(string n) {
        name = n;
    }

    void rentVehicle(Vehicle* v) {
        cout << "\n" << name << " is trying to rent a car..." << endl;
        rs.rentVehicle(v);
    }

    void returnVehicle(Vehicle* v) {
        cout << "\n" << name << " is returning a car..." << endl;
        rs.returnVehicle(v);
    }
};
```

Object Oriented Programming:

```
// Main - Array of base class pointers with dynamic memory
int main() {
    // Array of base class pointers with dynamic memory allocation
    Vehicle* fleet[3];
    fleet[0] = new Car(101, "Toyota", "Corolla");
    fleet[1] = new Car(102, "Honda", "Civic");
    fleet[2] = new Car(103, "Audi", "r8");

    // Display all cars
    cout << "===== Available Cars in Fleet =====" << endl;
    for (int i = 0; i < 3; i++) {
        fleet[i]->displayInfo();
        cout << " | Status: " << (fleet[i]->isAvailable() ? "Available" :
"Rented") << endl;
    }

    // Customers
    Customer c1("Ali");
    Customer c2("Faarid");

    // Rent operations
    c1.rentVehicle(fleet[0]); // Ali rents Toyota Corolla
    c2.rentVehicle(fleet[0]); // Sara tries to rent same car (not available)
    c2.rentVehicle(fleet[1]); // Sara rents Honda Civic

    // Return operations
    c1.returnVehicle(fleet[0]); // Ali returns Toyota Corolla
    c2.rentVehicle(fleet[0]);   // Sara rents it again

    // Display final status
    cout << "\n===== Final Fleet Status =====" << endl;
    for (int i = 0; i < 3; i++) {
        fleet[i]->displayInfo();
        cout << " | Status: " << (fleet[i]->isAvailable() ? "Available" :
"Rented") << endl;
    }

    // Free dynamic memory
    for (int i = 0; i < 3; i++)
        delete fleet[i];

    return 0;
}
```

Made By: Faarid Amir Shaikh CT-25276 Section: F

Output:

```
6_Q3 }
===== Available Cars in Fleet =====
Car ID: 101 | Brand: Toyota | Model: Corolla | Status: Available
Car ID: 102 | Brand: Honda | Model: Civic | Status: Available
Car ID: 103 | Brand: Audi | Model: r8 | Status: Available

Ali is trying to rent a car...
Car rented successfully -> Car ID: 101 | Brand: Toyota | Model: Corolla

Faarid is trying to rent a car...
Sorry, this vehicle is not available for rent!

Faarid is trying to rent a car...
Car rented successfully -> Car ID: 102 | Brand: Honda | Model: Civic

Ali is returning a car...
Car returned successfully -> Car ID: 101 | Brand: Toyota | Model: Corolla

Faarid is trying to rent a car...
Car rented successfully -> Car ID: 101 | Brand: Toyota | Model: Corolla

===== Final Fleet Status =====
Car ID: 101 | Brand: Toyota | Model: Corolla | Status: Rented
Car ID: 102 | Brand: Honda | Model: Civic | Status: Rented
Car ID: 103 | Brand: Audi | Model: r8 | Status: Available
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 09>
```

Question#04:

Code:

```cpp
#include <iostream>
#include <string>
using namespace std;

// Abstract Base Class
class EncryptionTechnique {
protected:
    string message;

public:
    EncryptionTechnique(string msg) {
        message = msg;
    }
```

Object Oriented Programming:

```
    virtual void encrypt() = 0; // Pure virtual function
};

// Derived Class: EncryptionTechnique1
// Converts each alphabet to its ASCII code
class EncryptionTechnique1 : public EncryptionTechnique {
public:
    EncryptionTechnique1(string msg) : EncryptionTechnique(msg) {}

    void encrypt() override {
        cout << "Technique 1 Encrypted Message: ";
        for (int i = 0; i < message.length(); i++) {
            cout << (int)message[i];
        }
        cout << endl;
    }
};

// Derived Class: EncryptionTechnique2
// Converts each alphabet to its ASCII code and adds 2
class EncryptionTechnique2 : public EncryptionTechnique {
public:
    EncryptionTechnique2(string msg) : EncryptionTechnique(msg) {}

    void encrypt() override {
        cout << "Technique 2 Encrypted Message: ";
        for (int i = 0; i < message.length(); i++) {
            cout << (int)message[i] + 2;
        }
        cout << endl;
    }
};

int main() {
    string input;
    cout << "Enter message to encrypt: ";
    cin >> input;

    EncryptionTechnique* e1 = new EncryptionTechnique1(input);
    EncryptionTechnique* e2 = new EncryptionTechnique2(input);

    e1->encrypt();
    e2->encrypt();
```

Made By: Faarid Amir Shaikh CT-25276 Section: F

```
    delete e1;
    delete e2;

    return 0;
}
```

Output:

```
6_Q4 }
Enter message to encrypt: Sir Imran
Technique 1 Encrypted Message: 83105114
Technique 2 Encrypted Message: 85107116
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 09>
```